

Lecture 7: Memory Management

Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se



UNIVERSITY OF
GOTHENBURG

What to read (main textbook)

- Chapter 3.1, 3.2, 3.7 (up to 3.7.1 - excluded)

(extra facultative reading: 8.1-8.5 from Silberschatz Operating System Concepts)

Objective

- Discuss how different types of memory are used
- Discuss how information is continuously moved between “slow memory” and “fast memory” (accessed directly by the CPU)
- Discuss how and why user/programmer/program view of memory and physical memory are kept separated

Agenda

- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Agenda

- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

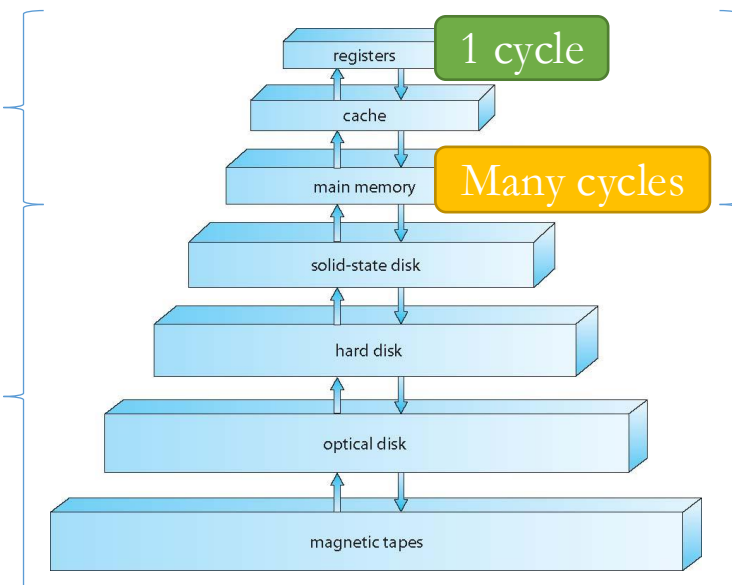
Background and basic hardware (I)

Goal of the OS: maximize CPU utilization → Run multiple processes concurrently / in parallel → Share memory!

We first have to move it up here!



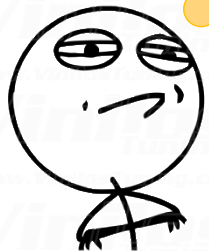
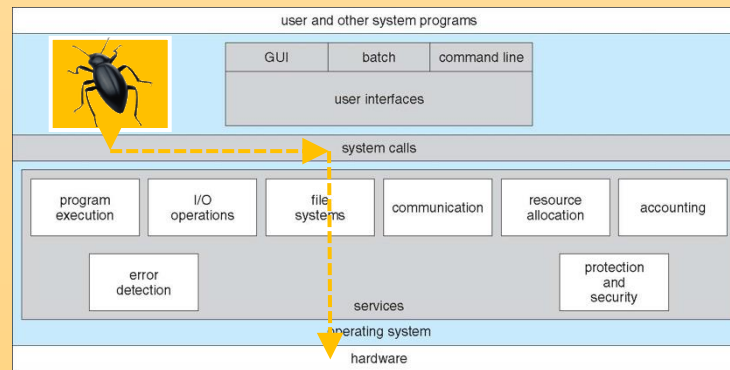
If some data the CPU wants to access is here



Can be accessed directly by the CPU

Cannot be accessed directly by the CPU

Background and basic hardware (II)

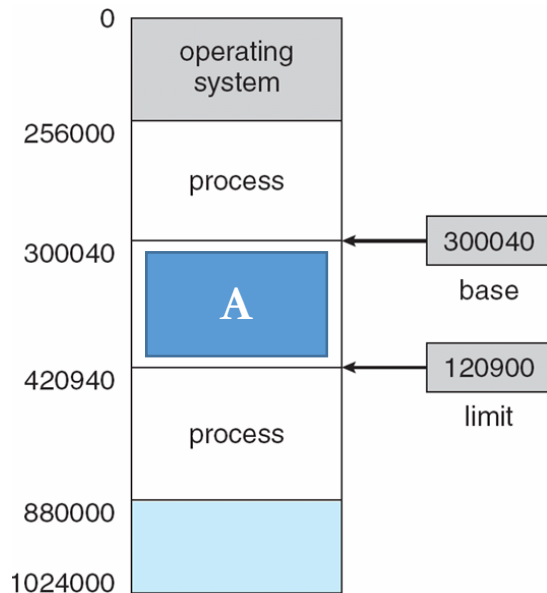


Need for fast CPU access to registers / caches and main memory?
→ hardware support without OS interaction
CHALLENGE: how to ensure correctness?

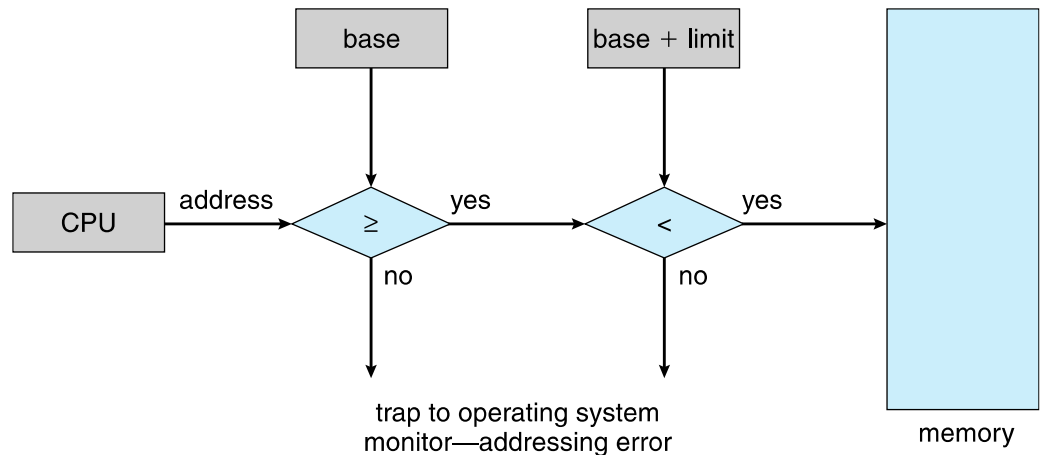
Agenda

- Introduction
- **Base and limit registers**
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

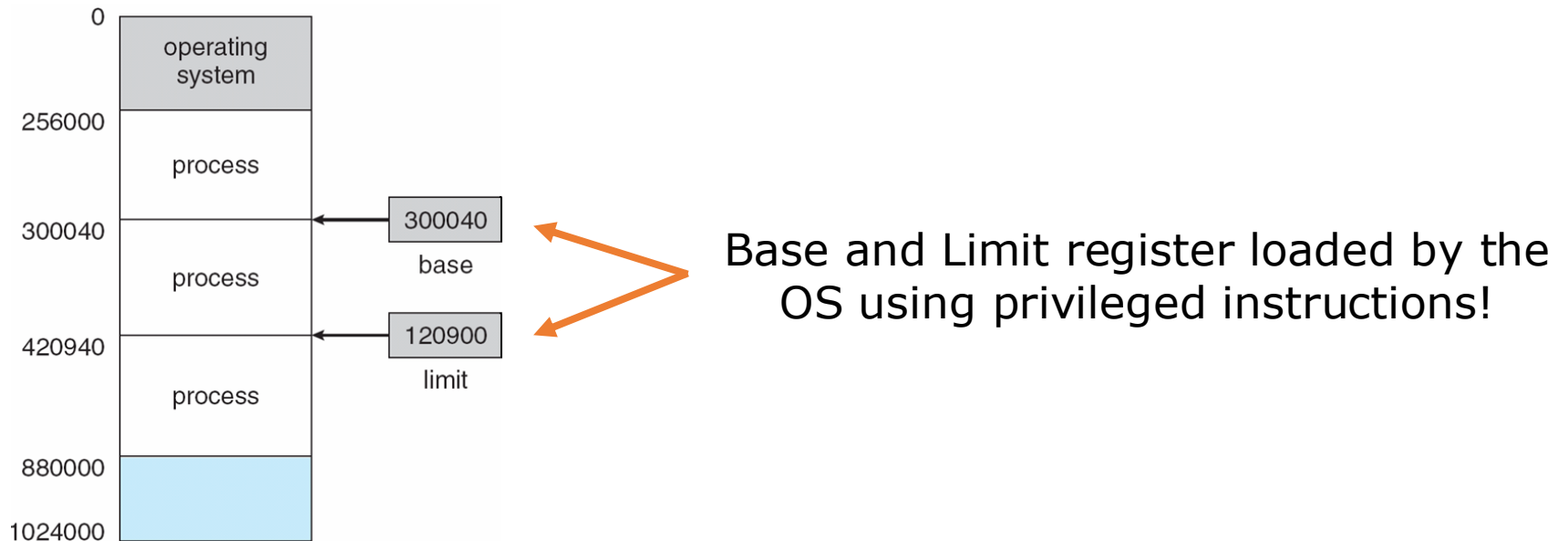
Base and Limit register (I)



- For each process, the CPU maintains two registers:
 - Base register: first address that can be accessed by the process
 - Limit register: number of addresses (starting from the base register) that can be accessed by the process
- Example: process A can only access addresses from 300040 to 420940



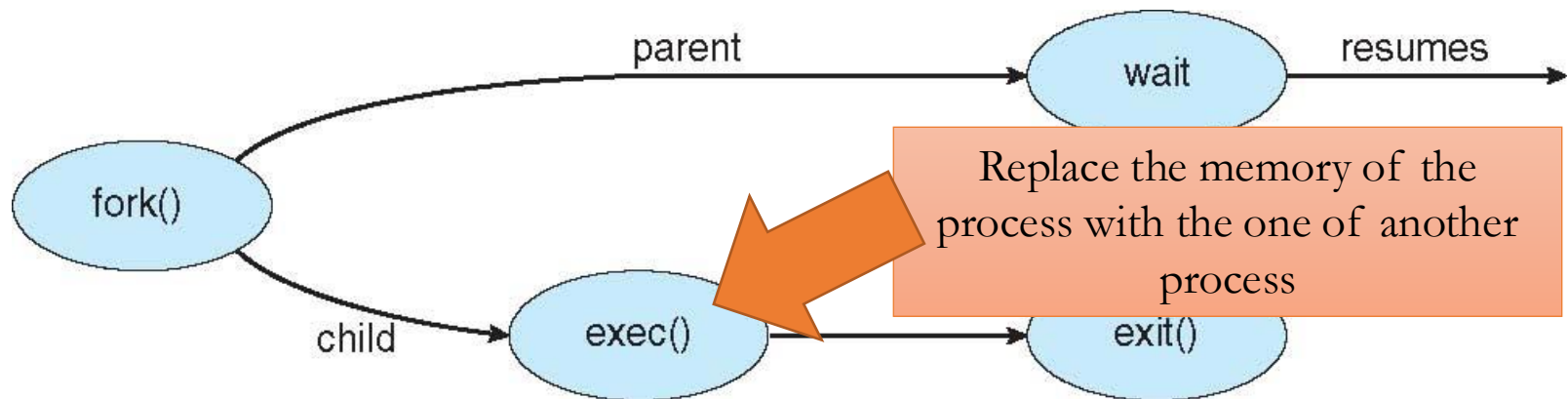
Base and Limit register (II)



Base and Limit register (III)

The operating system can access the memory of the processes without restrictions.

We saw (at least) one example of this in the previous lectures, when?



Agenda

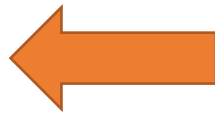
- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Address binding

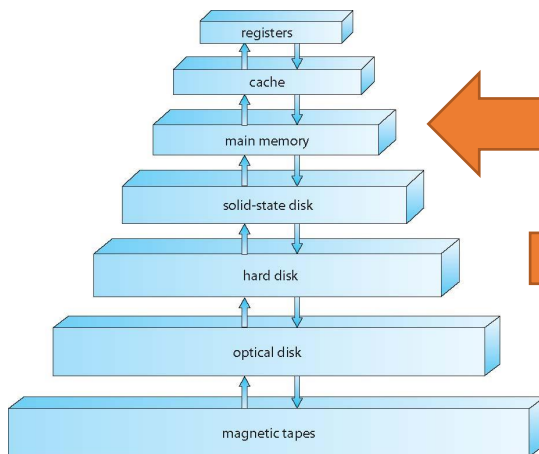
...

$a = b + 2$

...



Where do we store a?



Once we load it into memory, we know the address



As long as the program is not loaded in memory, we do not know the address

Logical/Virtual vs Physical address space (i)

Logical/Virtual address

...
a = b + 2
...

→ First address of the program 00000

→ Address of variable a 346

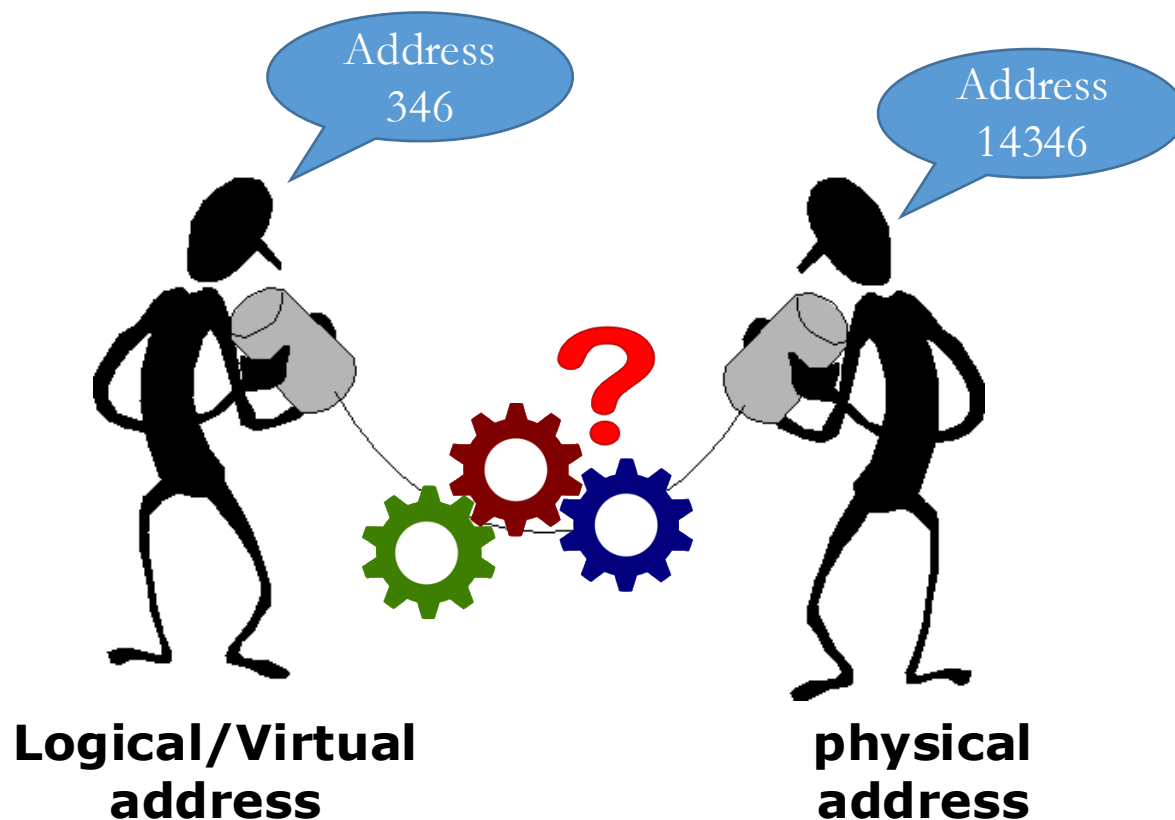
Physical address (main memory)

...
a = b + 2
...

→ First address of the program 14000

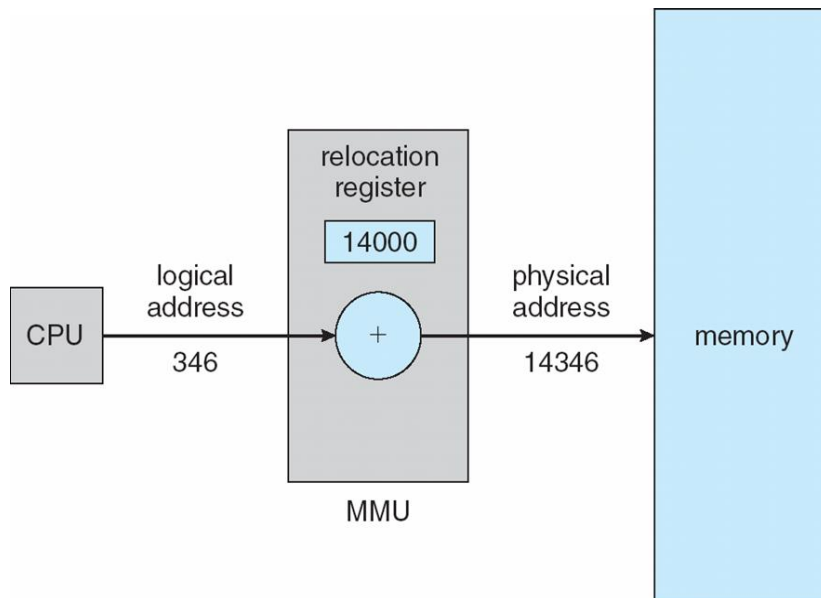
→ Address of variable a 14346

Logical/Virtual vs Physical address space (ii)



How to convert between logical/virtual addresses to physical addresses?

Memory-Management Unit (MMU) – Relocation Register

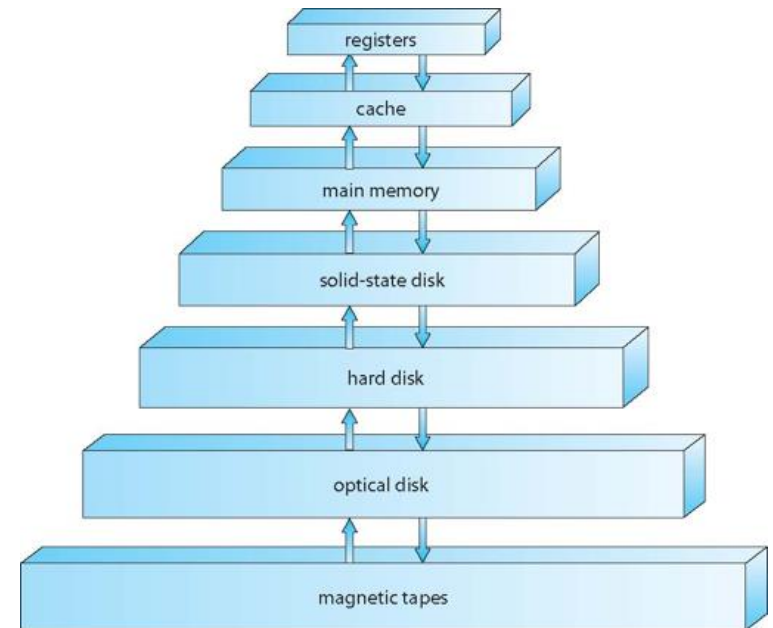


Logical/Virtual addresses $[0, \text{max}]$

Physical addresses $[R+0, R+\text{max}]$

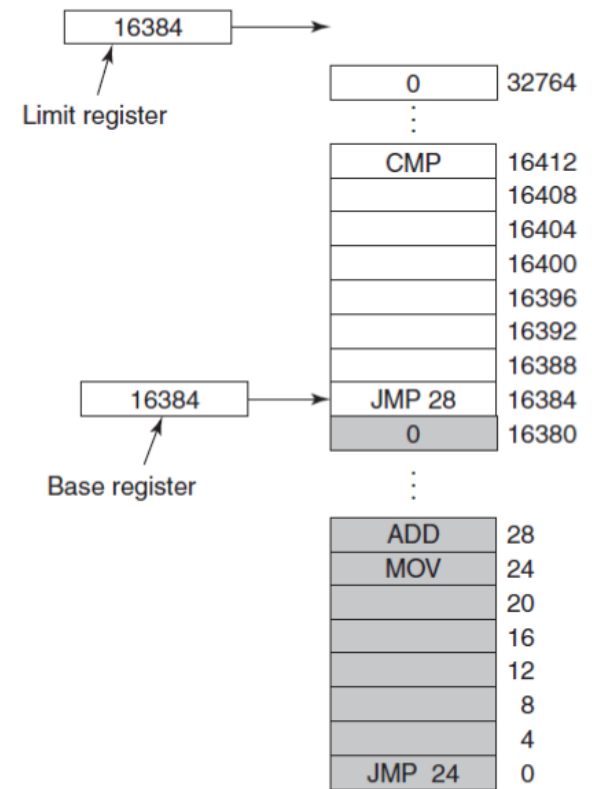
Which of these memories can be accessed directly by the CPU?

1. Registers, cache and main memory
2. Registers, cache, main memory and disk (solid-state)
3. Registers, cache, main memory and disk (any)



The base and limit registers in the figure contain the same value. Is it OK?

1. Yes, it is.
2. No, it is not.

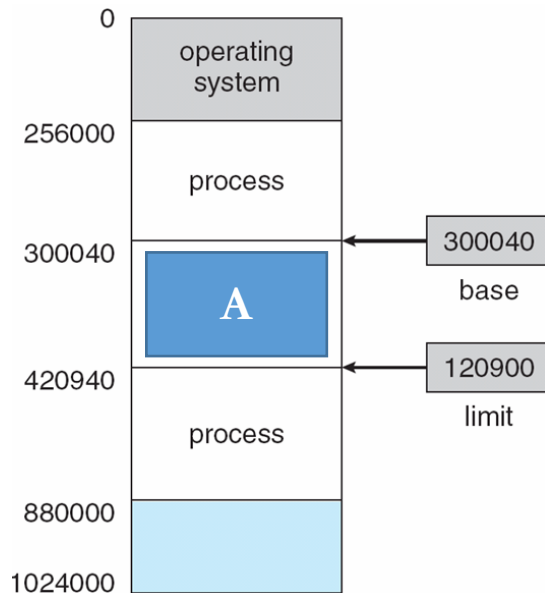


(c)

Agenda

- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- **Dynamic loading and linking optimizations [self reading]**
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Optimizations – Dynamic loading



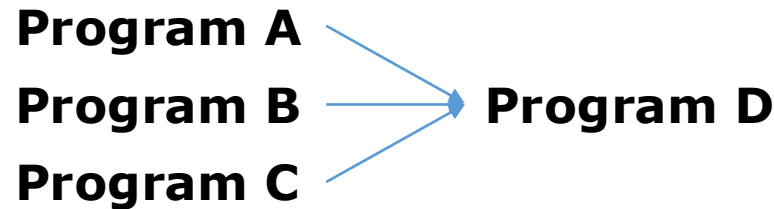
We assumed so far the entire program A is loaded

If A is large and parts of it are almost never used → slower to load

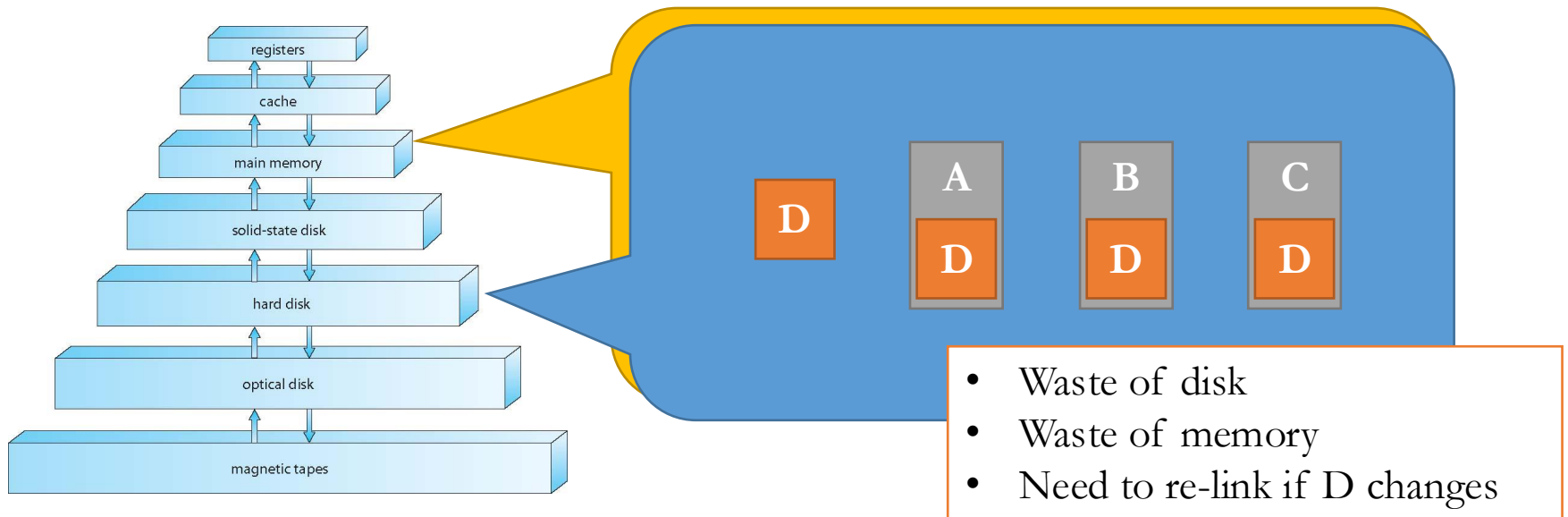
With dynamic loading, routines of a program can be loaded only when needed

Optimizations – Dynamic Linking (i)

Use library from

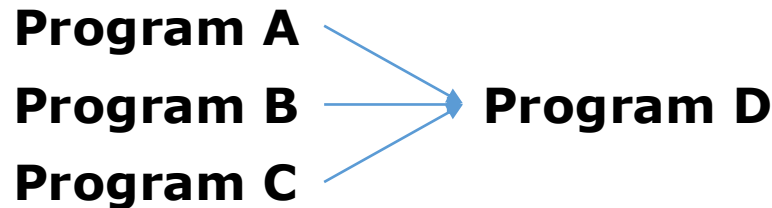


Static linking

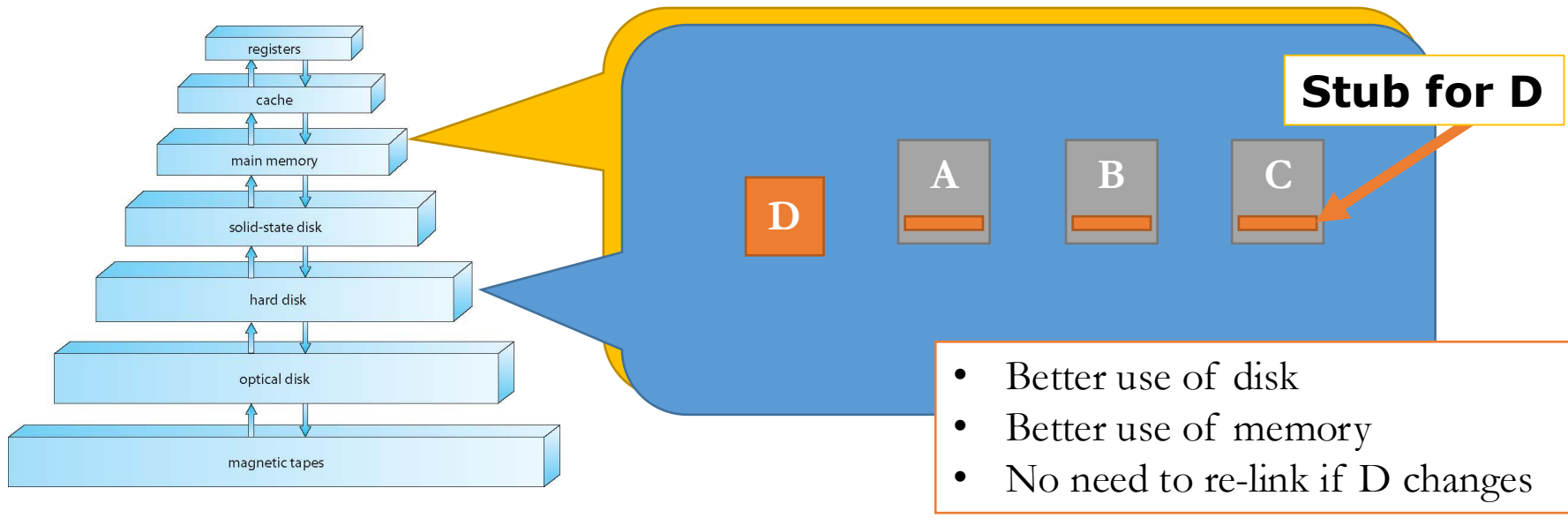


Optimizations – Dynamic Linking (ii)

Use library from



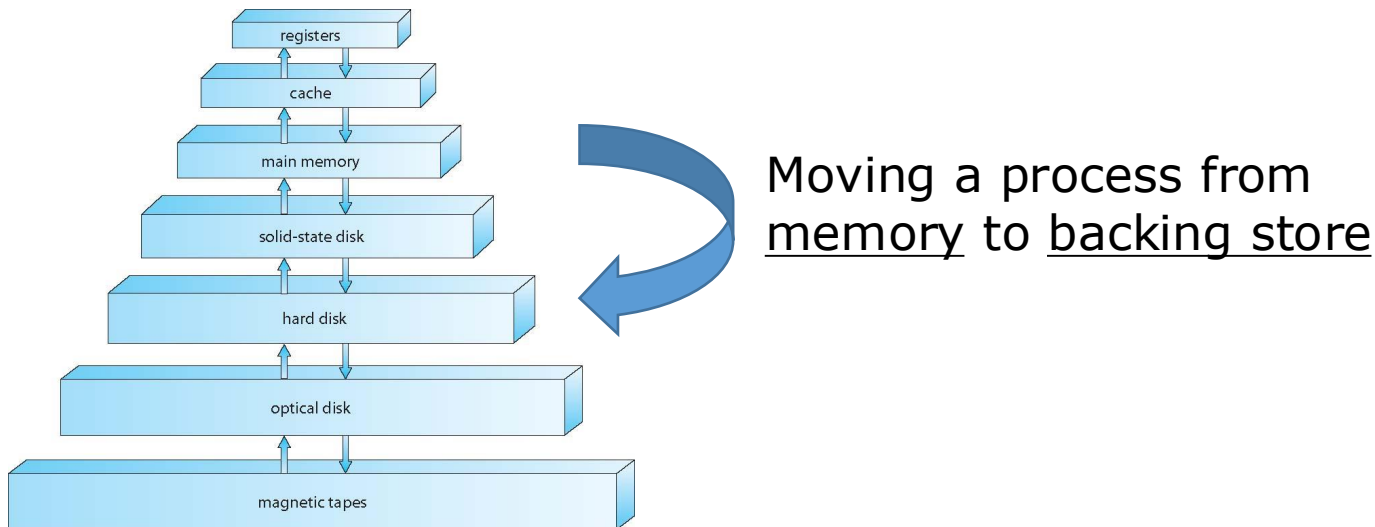
Dynamic linking



Agenda

- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- **Swapping**
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Swapping (i)

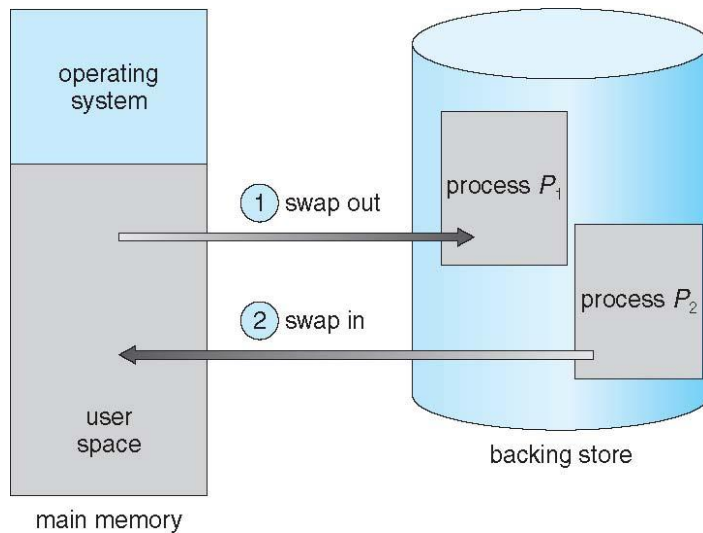


Why?

$\text{size}(\text{all logical addresses}) > \text{size}(\text{real physical memory})$

Swapping (ii)

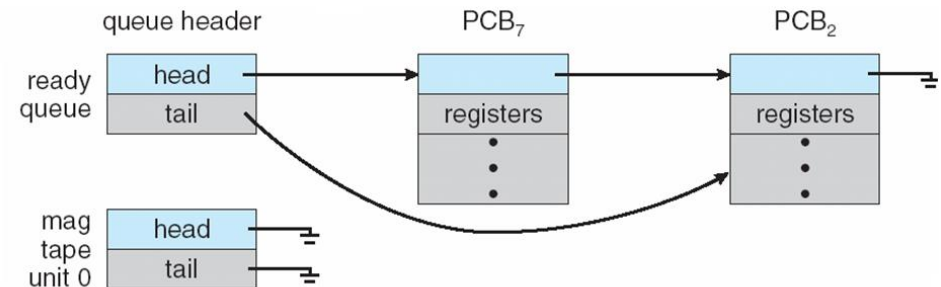
- Basic idea:



Questions:

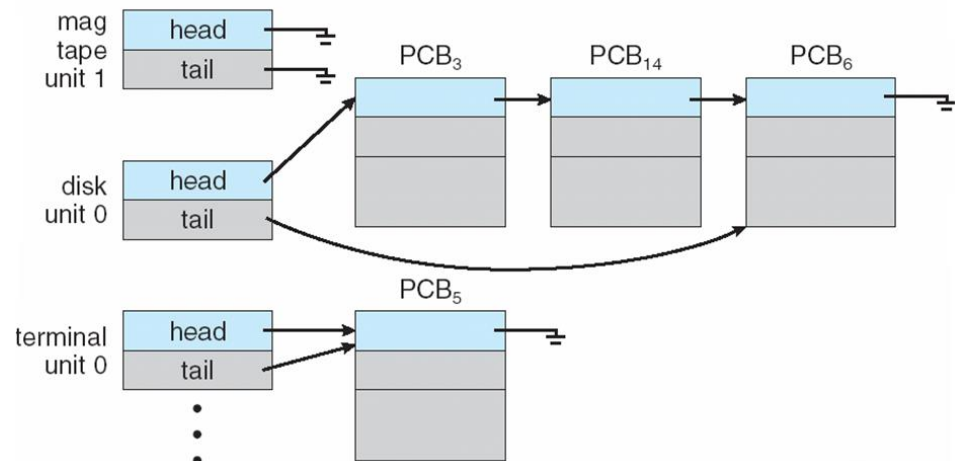
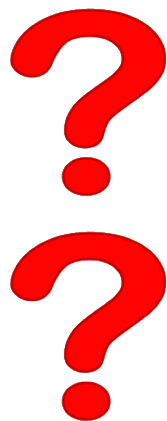
- Which processes can we swap?
- How much time does it take?
- When does it make sense to use swapping?

Swapping - Which processes can we swap?



Depends on the pending I/O...

Can the I/O results be buffered at the OS?



Swapping – How much time does it take?

- Process in memory 100 MB
- Disk transfer rate 50 MB/s

→ $100 \text{ MB} / 50 \text{ MB/s} = 2 \text{ seconds}$

→ Swap out + Swap in = 4 seconds

(Notice: being optimistic! Disk writes/reads might be requested by other processes, interrupts will occur in the meantime, etc...)

It is slow and expensive (because write/read to disks are slower and more expensive than memory ones)!

Swapping – When does it make sense to use it?

- Not used “as it is”...
- Modified versions are used in UNIX, Linux and Windows
 - Swapping is activated when free memory goes below a certain threshold
 - Swapping is activated when free memory goes above a certain threshold
- Modified swapping (swap out / in only parts of the process) used in conjunction with Virtual Memory (next lecture...)

Agenda

- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- **Memory allocation**
 - Contiguous allocation
 - Segmentation
 - Paging

Memory allocation

- Memory contains
 - The operating system itself
 - The users' processes
- Challenge: how to allocate memory efficiently?
- We will see 3 cases
 - Contiguous allocation
 - Segmentation
 - Paging



Logical and physical
memory more and
more separated

Less fragmentation (better
memory utilization)

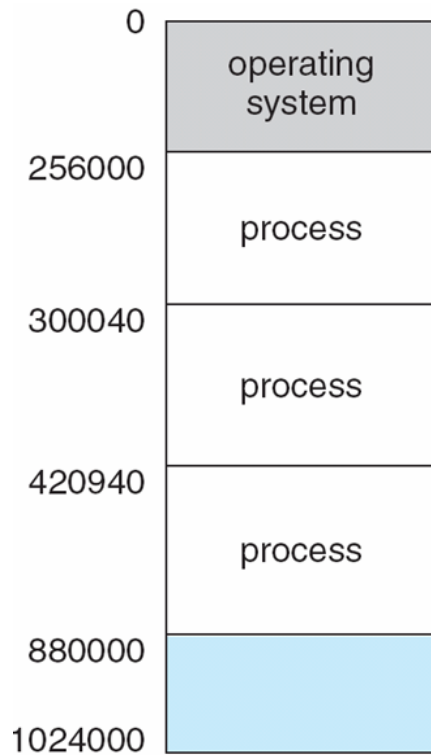
Protection

Faster loading
/ swapping

Agenda

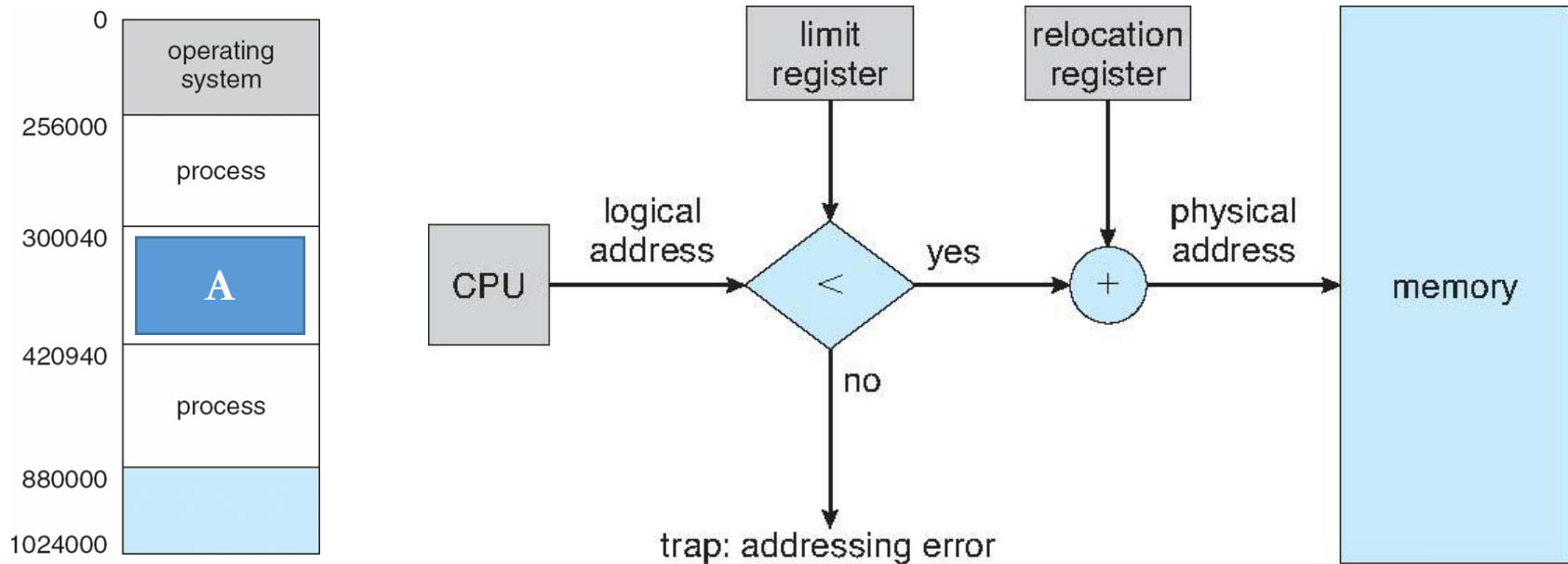
- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- Memory allocation
 - Contiguous allocation
 - Segmentation
 - Paging

Contiguous allocation

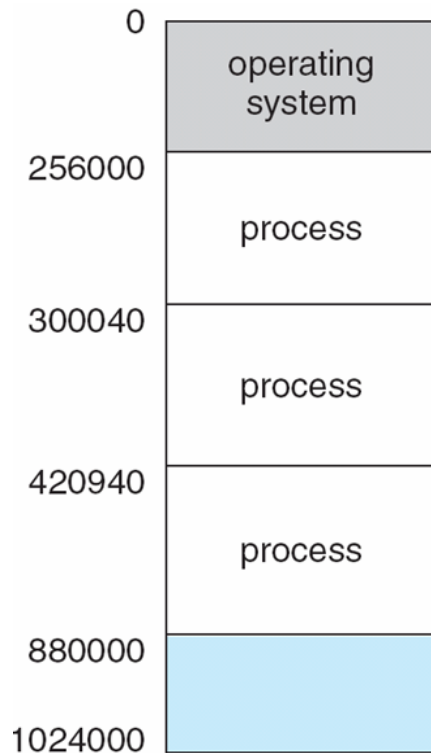


- Operating system at the bottom
- Processes one after the other
(each process contiguous to the previous)

Contiguous allocation – memory protection



Contiguous allocation - limitations



- When we terminate (or swap) a process
 - We create a **hole**
- The OS needs to keep track of holes to reuse them later on for other processes
- External fragmentation
$$\text{sum}(\text{all holes size}) > \text{size}(\text{a new process})$$

BUT

$$\text{size}(\text{of any hole}) < \text{size}(\text{a new process})$$

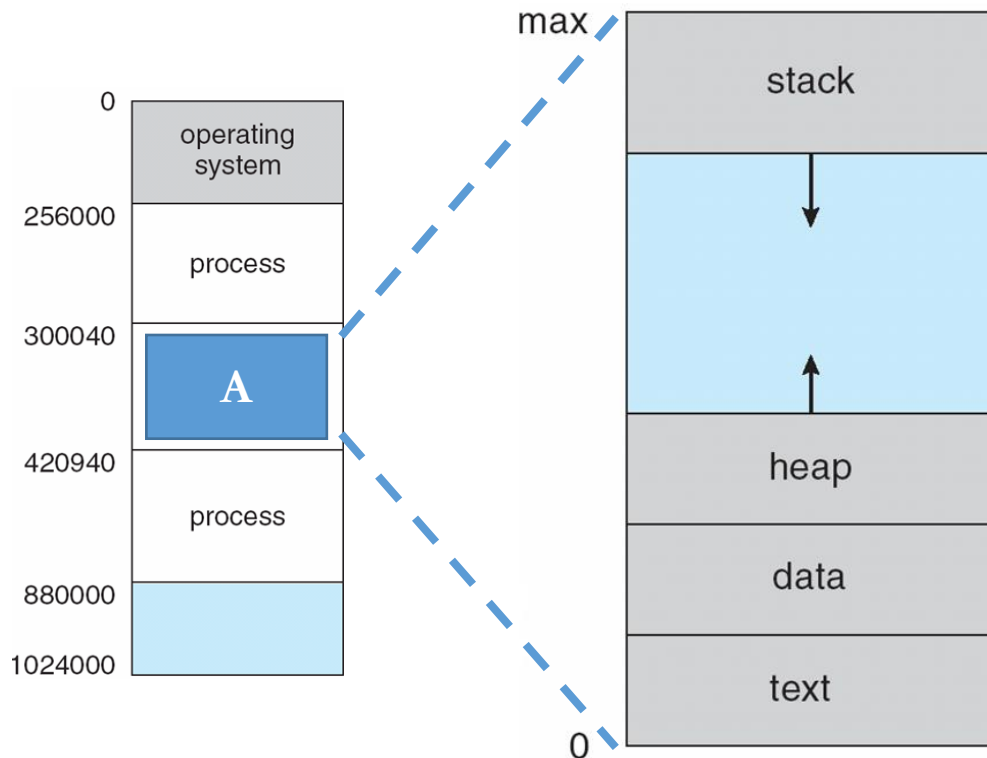
Contiguous allocation – which hole to assign to a new process?

- First fit → allocate the first hole that is big enough
- Best fit → allocate the smallest hole that is big enough
 - PROS: produces the smallest leftover hole
 - CONS: Search entire list (or keep list of holes sorted)
- Worst fit → allocate the largest hole
 - PROS: produces the largest leftover hole
 - CONS: Search entire list (or keep list of holes sorted)

Agenda

- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- **Memory allocation**
 - Contiguous allocation
 - Segmentation
 - Paging

Segmentation (i)

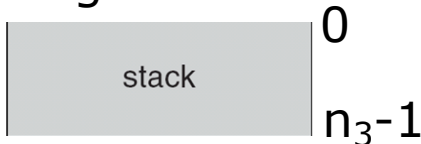


- So far we referred to the memory of a process as a single unidimensional space $[0, \text{max}]$
- Nevertheless, from the programmer/program perspective:
 - Different parts have different meaning, and each should be “protected from others”
 - Different parts have different and variable size

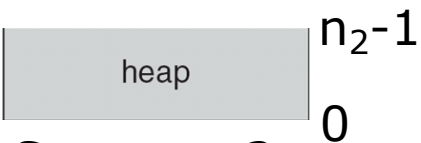
Segmentation (ii)

- With segmentation a memory position can be accessed not referring to its global position in $[0, \text{max}]$ but rather as a pair $\langle \text{segment}, \text{position in the segment} \rangle$

Segment 3

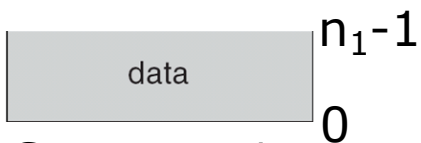


Read/write



Read/write

Segment 2



Read/write

Segment 1

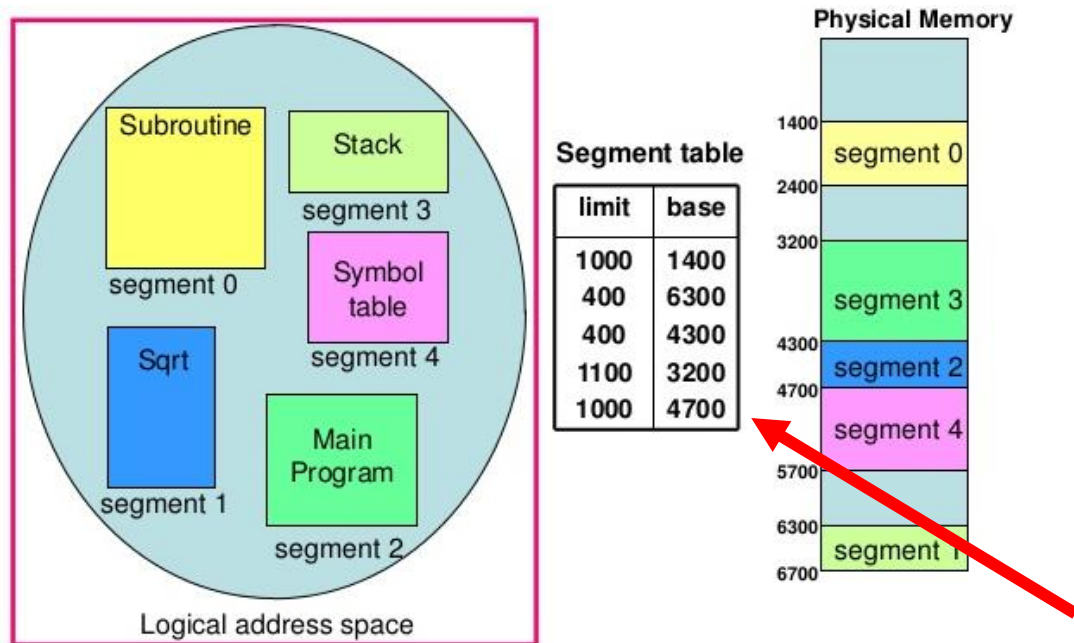


Execute

Segment 0

- Easier to check if an access to a certain section is actually reaching a different section
- Allows to have different permissions for the different segments!
- Different parts of the process can be at different locations in the physical memory!

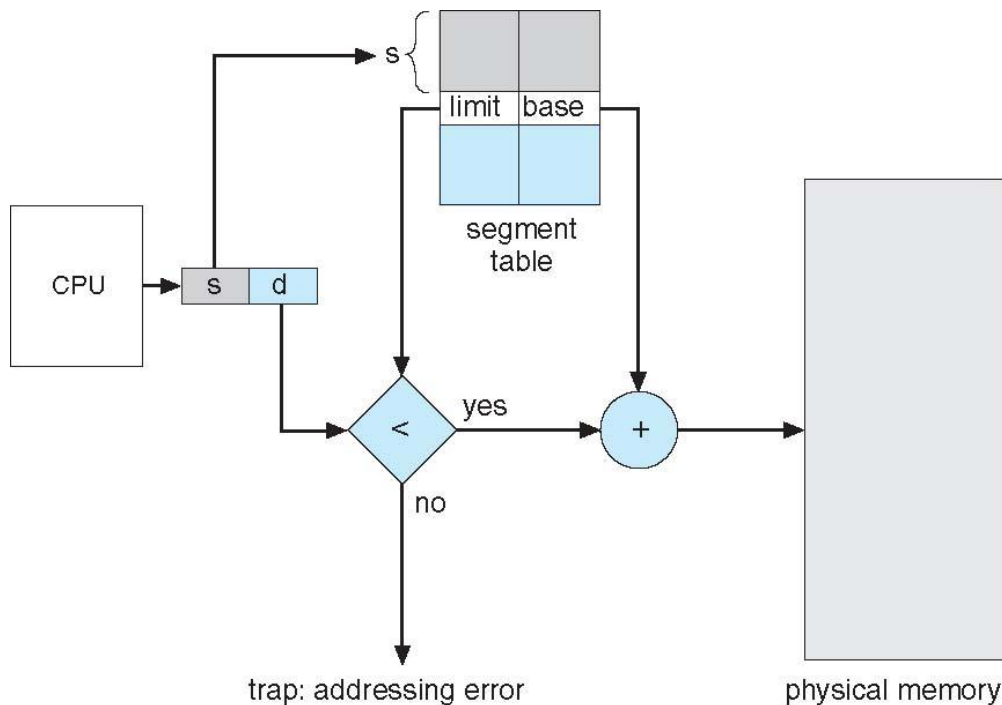
Segmentation – segment table



Need to maintain extra information to link logical address space / segments to the physical memory

Segmentation – hardware support

Hardware is able to do this conversion automatically (and check whether the process is trying to access memory outside the segment)



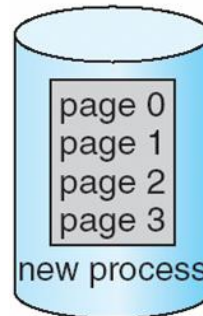
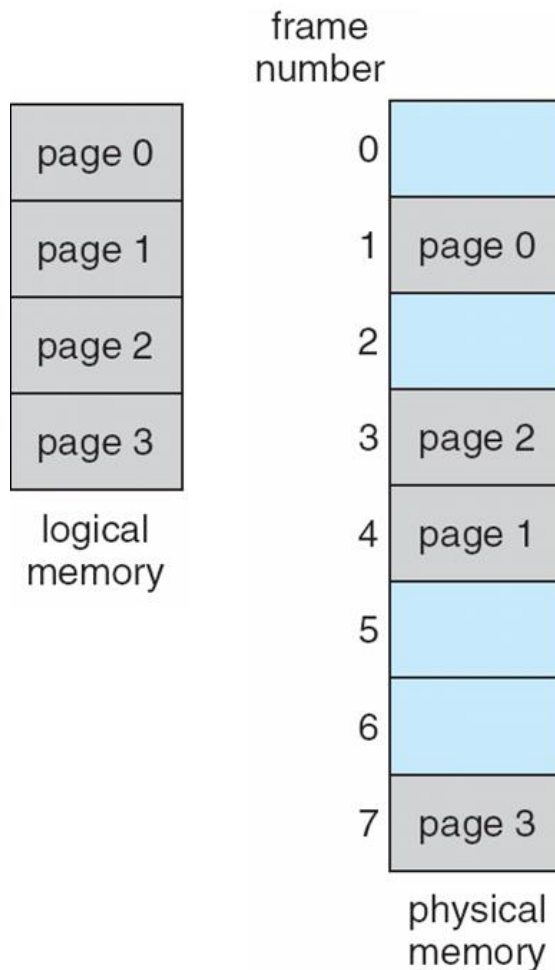
Agenda

- Introduction
- Base and limit registers
- Logical/Virtual vs Physical addresses
- Dynamic loading and linking optimizations [self reading]
- Swapping
- **Memory allocation**
 - Contiguous allocation
 - Segmentation
 - **Paging**

Paging

- Mechanism that allows for the physical address space of a process to be non-contiguous
- Used in most modern OSes (from mainframes to smartphones)
- Requires cooperation between OS and hardware (as common)

Paging – Basic idea



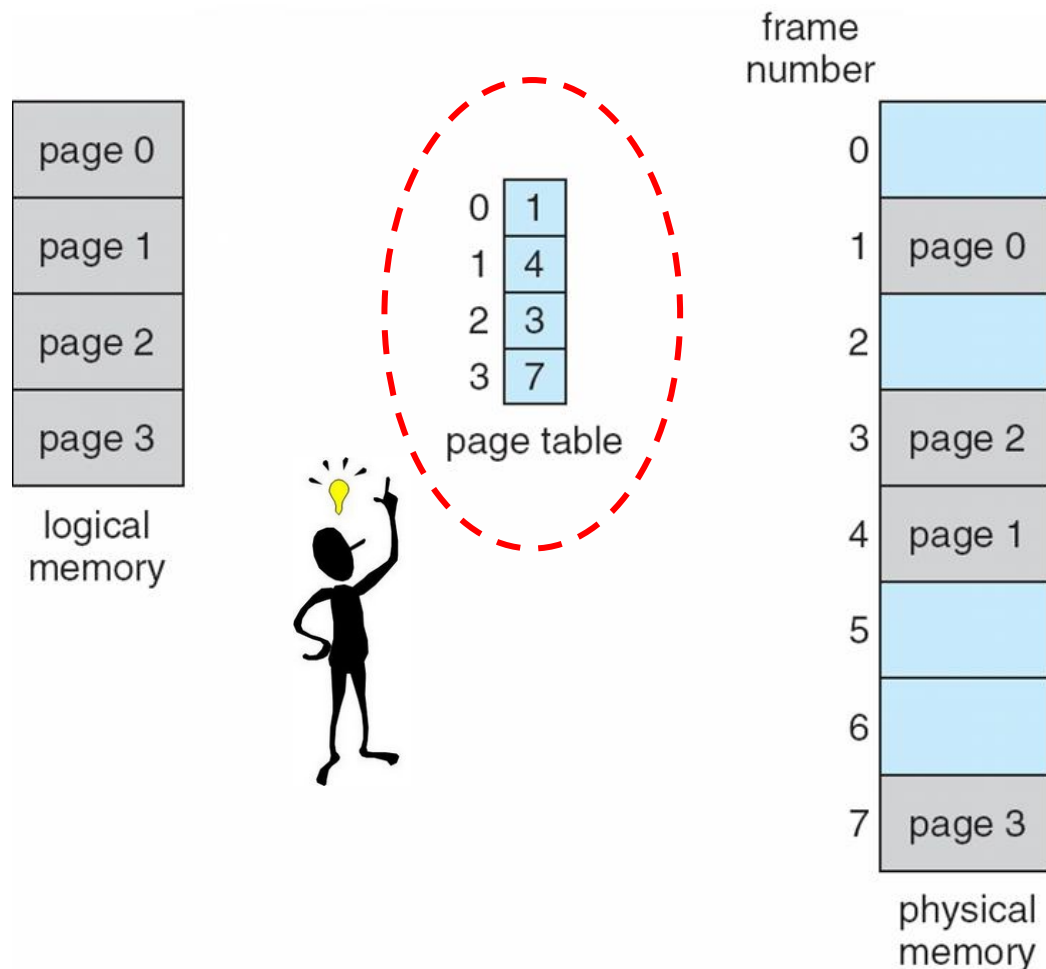
- Split logical memory in pages
- Split physical memory in frames
- Keep a program (disk) organized in pages

Page size = Frame size!



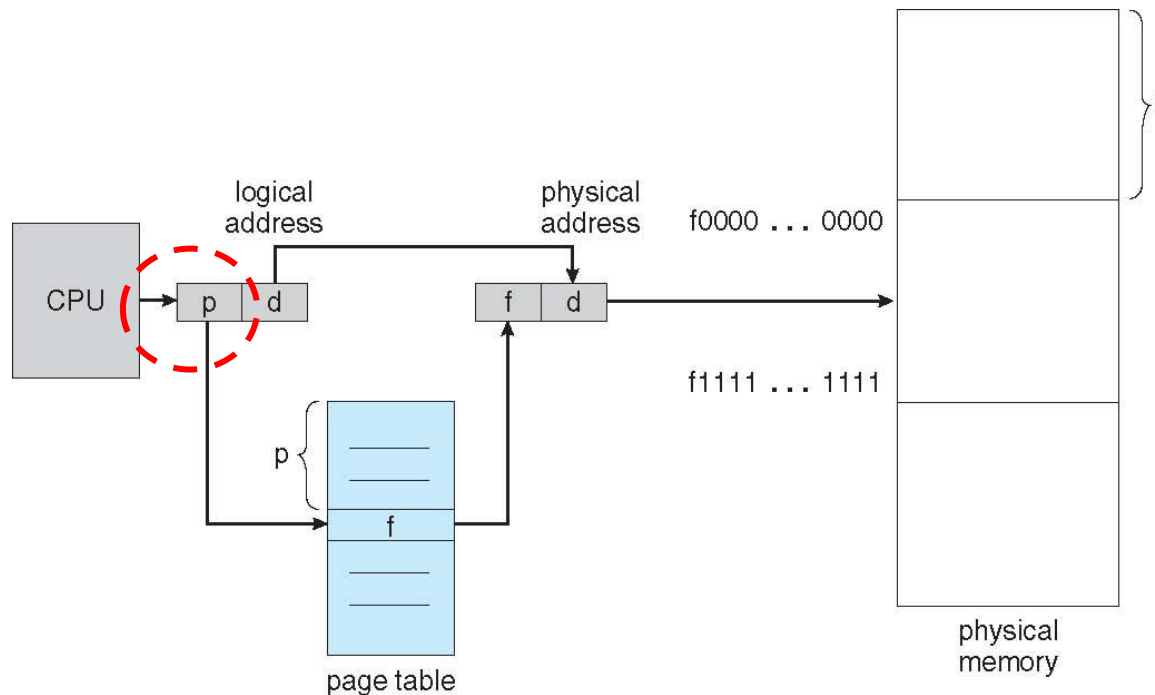
How do we keep track of where each page is?

Paging – the page table



- Page 0? Frame 1
- Page 1? Frame 4
- Page 2? Frame 3
- Page 3? Frame 7

Paging – hardware support

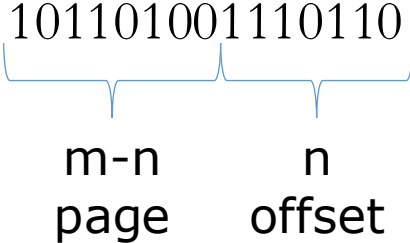


How does the CPU know which page it should access?

Similarly to Segmentation, we can access physical memory specifying page + offset

Paging – from logical address to physical address (i)

- If page/frame size is power of 2 $\rightarrow 2^n$
- If size of logical space is power of 2 $\rightarrow 2^m$

Logical address (binary) The diagram shows a binary logical address '101101001110110' with a blue bracket underneath. The bracket is divided into two parts: the left part, labeled 'm-n' and 'page', covers the first 8 bits '10110100'; the right part, labeled 'n' and 'offset', covers the last 6 bits '1110110'.

Paging – from logical address to physical address (ii)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i
	j
	k
8	m
	n
	o
	p
12	
16	
20	a
	b
	c
	d
24	e
	f
	g
	h
28	

physical memory

Page size = 2^2 bytes ($n=2$)

Size of logical space = 2^4 bytes ($m=4$)

→ logical space = 4 pages



Physical address of logical address 7?

0 1 1 1
page 1 offset 3

Frame 6 offset 3
→ $6 \times 4 + 3 = 27$

Paging - Fragmentation

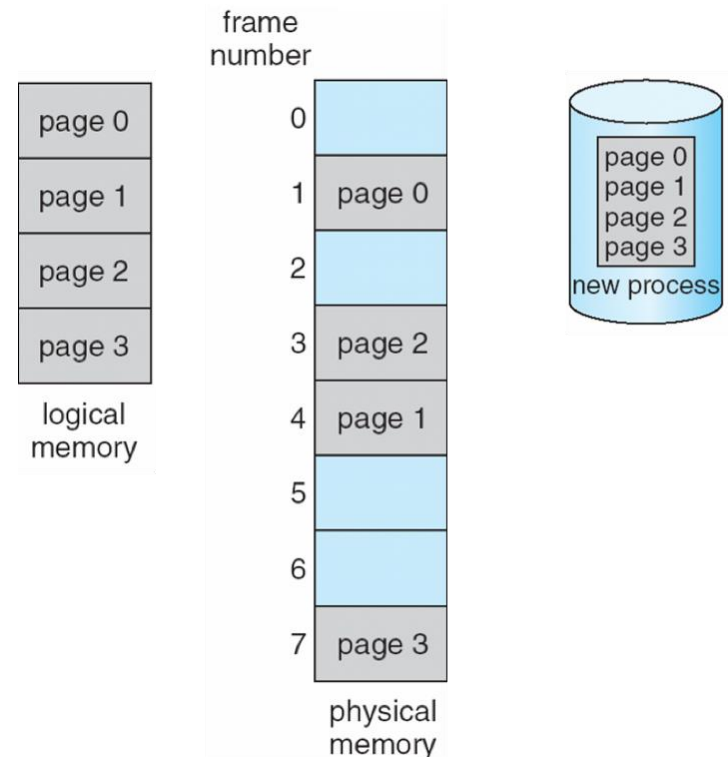
- No external fragmentation
 - Process needs a page? Can take any free frame
- Internal fragmentation
 - Last frame assigned to a process' page might be not full
 - On average, half frame is wasted for each process



- Small pages → less fragmentation / more overhead (e.g., page table size)
- Large pages → more fragmentation / less overhead

Paging suffers from...

1. External fragmentation
2. Internal fragmentation
3. Does not suffer fragmentation



Process p address space is 128K. There are 16K of physical memory available (page size is 1K).

1. The page table contains 128 entries
2. The page table contains 16 entries
3. The page table contains 8 entries